



Get Started with the CDM for Collateral

Module F – Advanced Features

Version 2.0 – June 2024

Click below to get started!

[Start](#)

F

This final module will introduce you to some of the more advanced concepts in the CDM that enable the model to be adopted to transform the processing of financial data.

1

In-bound Data – mapping & synonyms

2

Out-bound Data – reports, projection, APIs

3

Getting Involved

4

Contributing to the CDM

[Start](#)

- This final module addresses some Advanced Features in the CDM.
- As we have experienced the CDM so far, we have looked at it primarily as a model of financial products and events. But to be useful, the model has to be implemented to provide real and lasting value in a financial institution. So we will look at how the CDM is typically implemented into an existing ecosystem.
- First, we will look at ways of loading transactions, products and events into the CDM and the support provided for translating external data into the CDM model.
- Secondly, we will look at getting data out of the CDM and the features provided for reporting (i.e. generalised data exports) and projection (translating outbound data to new non-CDM formats and representations).
- Finally, in the very last two sections, we will look at what you can do to remain involved with the development of the CDM and, should you have the interest or need, how you can contribute to the model and its future.

F

1

In-bound Data – mapping & synonyms

[Start](#)

2

Out-bound Data – reports and projection

[Start](#)

3

Getting Involved

[Start](#)

4

Contributing to the CDM

[Start](#)

- The FINOS CDM Distribution includes a set of mappings to facilitate the translation of existing industry messages (based on open standards or proprietary ones) into the CDM.
- Mapping in Rune provides a mechanism for specifying how documents in other formats (e.g. FpML or ISDACreate) should be transformed into Rune documents. Mappings are specified as synonym annotations in the model.
- The CDM is mapped to a set of those alternative data representations using the Rune DSL “synonym” feature.
- Synonyms added throughout the model are combined to map the data tree of an input document into the output Rune document. The synonyms can be used to generate an Ingestion Environment, a java library which, given an input document, will output the resulting Rune document.
- Synonyms are specified on the attributes of data type and the values of enum types.

- Basic mappings specify how a value from the input document can be directly mapped to a value in the resulting Rune document. The mappings themselves can be inside the model definitions, or external in a dedicated declaration file.
- The simplest synonym consists of a single **value** where the value of the input attribute will be mapped to the associated Rune attribute. If both the input attribute and the Rune attribute are basic types (string, number, date, etc.), then the input value will be stored in the appropriate place in the output document. If they are both complex types, then the attributes will be compared against synonyms inside the corresponding Rune type.
- Internal synonyms are annotations on attributes of Rune types and the enumeration values of Rune Enums, for example, this shows partly how Acadia maps an enum for Legal Agreement publishers:

```
enum LegalAgreementPublisherEnum: <"The enumerated values to specify the legal agreement publisher.">
  [synonym AcadiaSoft_AM_1_0 value "PublisherEnum"]
  AFB <"Association Française des Banques.">
  TheFXCommittee <"The Foreign Exchange Committee">
  ISDA <"International Swaps and Derivatives Association, Inc.">
  [synonym AcadiaSoft_AM_1_0 value "ISDA"]
```

- External synonyms are defined separately using the synonym keyword; for example, this code snippet defines the mapping for certain collateral terms in FpML:

```
synonym source FpML_5_Confirmation_To_TradeState extends FpML
{
  CashCollateralValuationMethod:
    + applicableCsa
      [value "applicableCsa"]
    + cashCollateralCurrency
      [value "cashCollateralCurrency"]
    + cashCollateralInterestRate
      [value "cashCollateralInterestRate"]
}
```

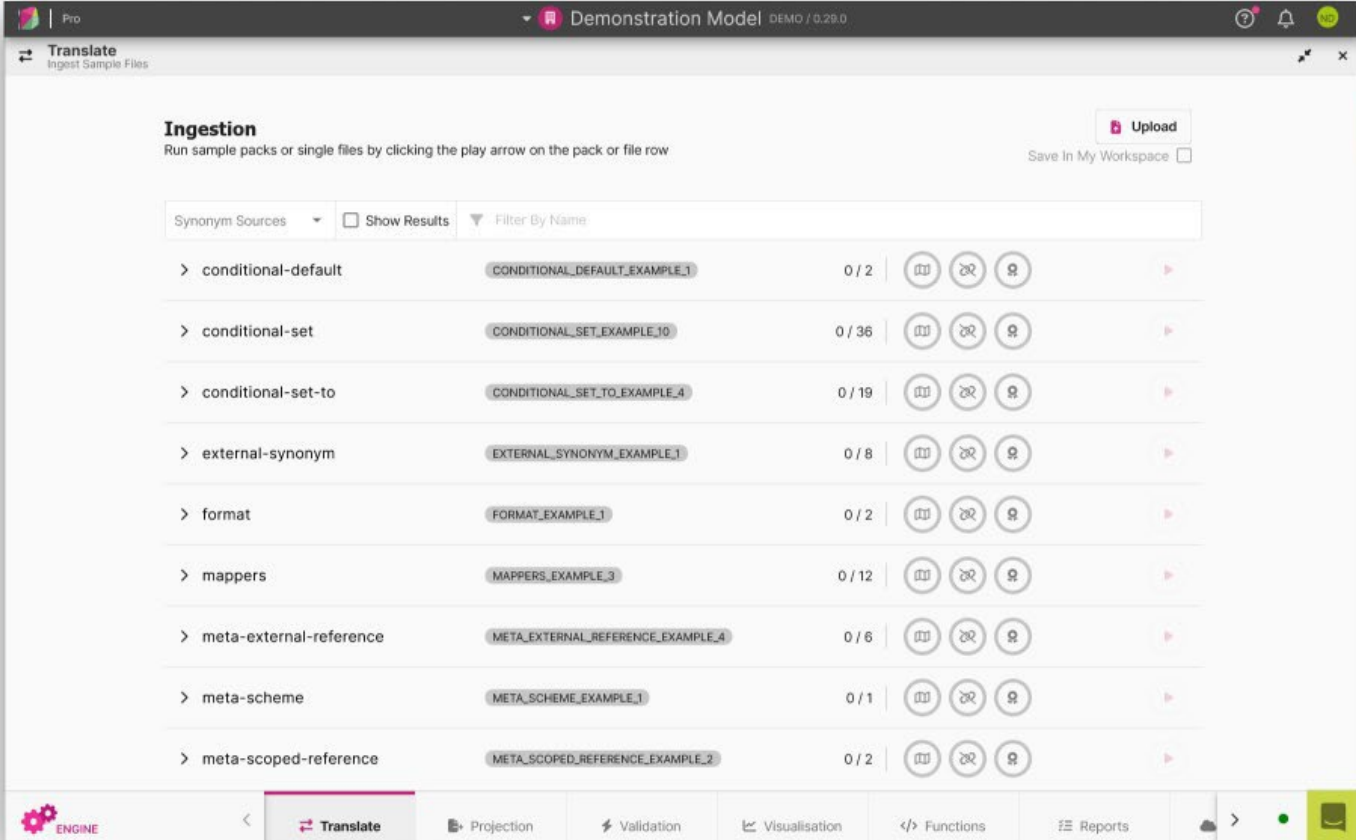
- The Rune DSL has many features that result in a powerful mapping capability.
- The [full documentation](#) describes this and we recommend referring to this when you need to build a new mapping capability for a new source – for example, your internal, proprietary application.
- Some advanced features include
 - **path** – to navigate through a XML structure to a specific attribute
 - **maps 2** – to resolve one to many mappings
 - **meta** – to map metadata schemes
 - **hint** – to bypass a layer of Rune without consuming an input attribute
 - **pattern** – to manipulate the format of data
 - Logic functions including conditionals and **when** clauses.

- The following set of synonym sources are currently in place for the CDM; these sources are listed as part of a configuration file in the CDM using a special **synonym source enumeration**, so that the synonym source value can be controlled when editing synonyms.
- FpML standard (**FpML_5_10**)
- FIX standard (**FIX_5_0_SP2**)
- ISO 20022 reporting standard (**ISO_20022**).
- DTCC trade matching confirmations and payment notifications (**DTCC_11_0** & **DTCC_9_0**)
- CME OTC clearing confirmation & submission (**CME_ClearedConfirm_1_17** & **CME_SubmissionIRS_1_0**)
- AcadiaSoft Agreement Manager (**AcadiaSoft_AM_1_0**)
- ISDA Create tool for Initial Margin negotiation (**ISDA_Create_1_0**)
- ORE XML Model (**ORE_1_0_39**)

- Beyond the features of the FINOS CDM Distribution, the third-party application **Rosetta Translate** enables the user to keep in control of a mapping layer between their existing systems, databases and message formats and the format underpinned by a model.
- The main artefact of the mapping layer is called the Translation Dictionary, which is created thanks to Rosetta Translate.
- Rosetta Translate is closely linked to another Rosetta product: Rosetta Ingest. Rosetta Ingest offers an universal API to ingest and normalise the user data, by leveraging on the mapping layer that is created thanks to Rosetta Translate.
- In this section, you will learn about:
 - Ingestion and Projection
 - Exploring the file
 - Uploading a file
 - Running Ingestion
 - Viewing the results.

- **Ingestion** is the process of converting an (electronic) input file into the format underpinned by a model.
- The input file can be one of different standard formats for electronic data storage and transport, such as XML or JSON.
- The model output can be:
 - An electronic file, using the same range of support formats as the input file (a.k.a. a *Serialised* model document).
 - A model object that computer code can be directly executed on (a.k.a. a *Deserialised* model object).
- The Serialised view is useful to display the model output into an interface, in a way that a user can browse through and understand. This is the view that is used when demonstrating how Rosetta Translate really works.
- The De-Serialised object is not meant to be readable.
- Conversely, **Projection** is the process of converting a model object into an (electronic) file with a particular targeted format (for example another data standard such as ISO 20022).

- The FINOS CDM Distribution includes a number of test files, in test packs, which can be used as samples to demonstrate the model and also as test files to assure the accuracy of implementations of the model.
- These test packs can be [downloaded from GitHub](#) and are also available within Rosetta Translate and can be imported into Ingestion.



The screenshot shows the 'Ingestion' section of the Rosetta Translate interface. The title bar indicates 'Pro', 'Demonstration Model', and 'DEMO / 0.29.0'. The main heading is 'Ingestion' with a subtext 'Run sample packs or single files by clicking the play arrow on the pack or file row'. There is an 'Upload' button and a 'Save In My Workspace' checkbox. A 'Filter Bar' is located at the top left of the list, containing 'Synonym Sources', 'Show Results', and 'Filter By Name'. The list contains several test packs, each with a group icon (a right-pointing chevron), a name, a progress indicator, and three circular icons (a play button, a refresh button, and a search button). The packs listed are: conditional-default, conditional-set, conditional-set-to, external-synonym, format, mappers, meta-external-reference, meta-scheme, and meta-scoped-reference. A 'File Explorer' label is on the left side of the interface.

File Explorer in Rosetta Ingestion

- To view the test pack, open [Rosetta](#) and navigate to **Translate** on the bottom taskbar.
- The tests are organised in expandable groups, click on the group icon > to expand each of the groups to find the tests.
- You can read the [full documentation](#) on the features and navigation in this screen.

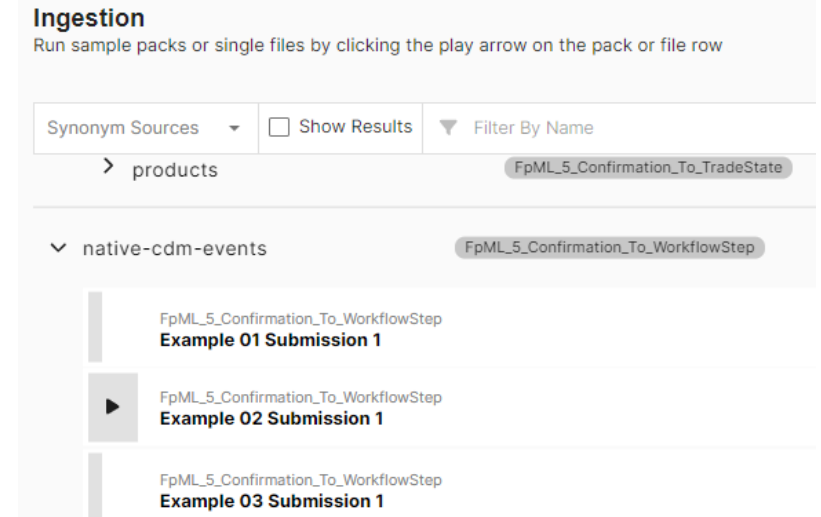
- To run an ingestion, navigate to the test file of your choice and then a Play button will appear next to the file. Click on the button to ingest that file.
- After a few moments, the file will be processed, and you will be able to view the mapping status:



- Hover over the status icons to see the detail:

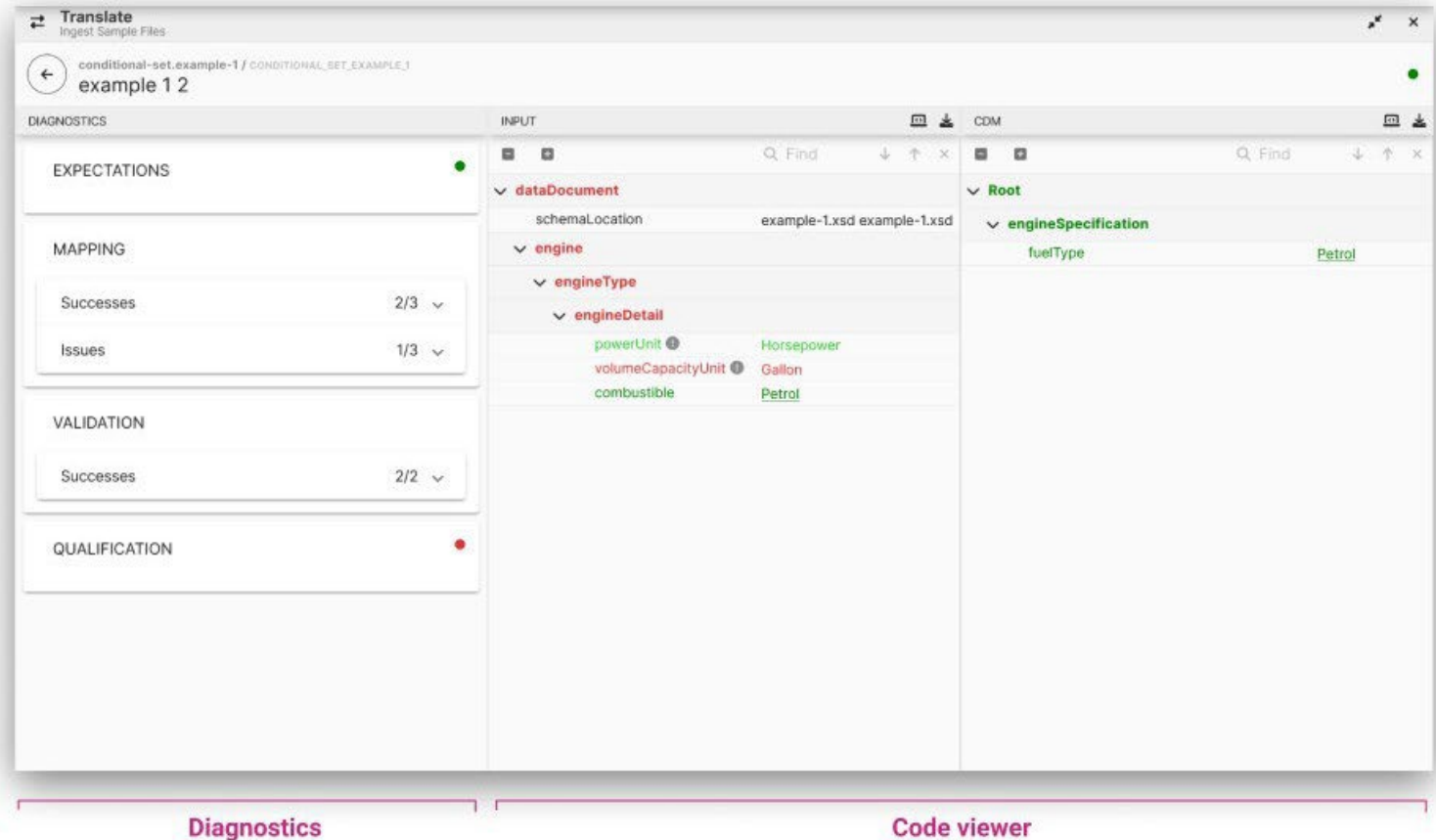


Detailed Diagnostic Summary



- Success:** The actual vs. expected success count for that result type.
- Failure:** The actual vs. expected failure count for that result type.
- Excluded:** The actual vs. expected excluded count for that result type.

- The results of the Ingestion can be viewed by clicking on the test file row.
- The **Result Viewer** displays the results; it is split vertically into three panels, from left to right:
 - Diagnostics panel: this panel displays statistics (Mapping, Validation, Qualification) and other key information summarising the success (or failure) of ingesting the file.
 - Input File View and Model Output View panels: these panels allow the user to review how the system has ingested the input file. The Input File View corresponds to the input file used for Ingestion and the Model Output View corresponds to the output in the model format.



- The Input File View and the Model Output View have two view modes:
 - Code View: displays the file in its original mark-up format.
 - Formatted Document View: creates a tree structure which is colour-coded to indicate the result of the Ingestion process.

- As an example of inbound data ingesting into the CDM for a collateral document, let's look at the translations for **ISDA Create** documents.
- These translations use a ISDA Create JSON document as the input and loads this into the CDM.
- The translation mapping is coded in the CDM as synonyms; you can find them in the ISDA Foundations model in GitHub, Rosetta or your favourite tool; the namespace is `isda.mapping.createiq:synonym`

```
synonym source CreateIQ_1_0 extends CreateIQ
{
  AccessConditionsElections:
    + party
      [set to CounterpartyRoleEnum -> Party1 when "partyA_illegalit
      [set to CounterpartyRoleEnum -> Party2 when "partyB_illegalit
    + illegality
      [set to True when "partyA_illegality" = "applicable" , set to
      [set to True when "partyB_illegality" = "applicable" , set to
    + forceMajeure
      [set to True when "partyA_force_majeure" = "applicable" , set
      [set to True when "partyB_force_majeure" = "applicable" , set
    + taxEvent
      [set to True when "partyA_tax_event" = "applicable" , set to
      [set to True when "partyB_tax_event" = "applicable" , set to
    + taxEventUponMerger
      [set to True when "partyA_tax_event_upon_merger" = "applicabl
      [set to True when "partyB_tax_event_upon_merger" = "applicabl
    + creditEventUponMerger
      [set to True when "partyA_credit_event_upon_merger" = "applic
      [set to True when "partyB_credit_event_upon_merger" = "applic
```

F

1

In-bound Data – mapping & synonyms

[Complete](#)

2

Out-bound Data – reports and projection

[Start](#)

3

Getting Involved

[Start](#)

4

Contributing to the CDM

[Start](#)

- One of the applications of the Rune DSL is to facilitate the process of producing data that has been stored in the model – in particular, the large body of data reporting obligations that industry participants are subject to.
- The current industry processes to implement those outbound data and reporting are costly and inefficient.
- They involve translating pages of business documentation and legal language into business requirements which firms then have to code into their systems to support reporting.
- By contrast, a domain-model for the business process or activity provides standardised, unambiguous definitions for business data at the source. In turn, these business data can be used as the basis for the outbound process, such that it becomes unambiguous views of the business data.
- The Rune DSL allows to express those outbound rules as functional components in the same language as the model for the business domain itself. Using code generators, those functional rules are then distributed as executable code, for all industry participants to use consistently in their compliance systems.

- Rosetta Engine is a run-time execution engine that can run the functional logic contained in a model on physical data. Once this functional logic has been modelled, Rosetta Engine transforms it into an executable data pipeline using the source code that is automatically generated from that model.
- The Rosetta Engine is provided with a user interface in the Rosetta web application and also as an API. The API takes model objects as inputs, executes the data pipelines and returns the output. This API can be deployed either on cloud or on premise.
- Main characteristics:
 - Generic: Rosetta Engine can transform any data processing model into a physically executable data pipeline. The engine is made available under a comprehensive set of programmatic languages.
 - Live: Rosetta Engine can execute a data pipeline in real-time as user define their data processing model. This enables a test-driven approach where a model is developed together with a set of test scenarios.
 - Scalable: Rosetta Engine scales based on the number of data objects that need to be processed.

- Generically, the Rune DSL allows to specify any report using three types of rules:
 - timing - when to report;
 - eligibility - whether to report; and
 - field - what to report.
- A report is associated to an authoritative body and to the corpus(es) in which those rules are specified. Usually, but not necessarily, the authority that mandates the rules also supervises their application and collects the data. Timing, eligibility and field rules translate into obligations of “timing, completeness and accuracy” of reporting, as often referred to by supervisors.

- A report is specified using the following syntax:
- The report type itself is defined as a data type component whose attributes are the reportable fields.
- Each attribute can be associated to a reporting rule containing the logic for extracting or calculating that field, using the rule Reference keyword.
- Check the [documentation](#) for further details on the Rune DSL modelling language to codify the structure of outbound data.

```
report <Authority> <Corpus1> <Corpus2> <...> in <TimingRule>  
  from <InputType>  
  when <EligibilityRule1> and <EligibilityRule2> and <...>  
  with type <ReportType>
```

```
type <ReportType>:  
  <field1> <Type1> (x..y)  
  [ ruleReference <RuleName1> ]  
  <...>
```

- Projection is the name given to the generation of outbound data from the CDM. Like Reporting, the specification of the outbound data and the mapping is codified in the Rune DSL.
- Projection uses the Rosetta Engine to invoke the rules on a certain data input to generate the conformant output.
- Similar to Translate, test packs and uploaded data can be passed to the Engine to translate.

Projection

Projection: Project_EsmaEmirTradeReportTolso20022 Test Pack: Equity Synced

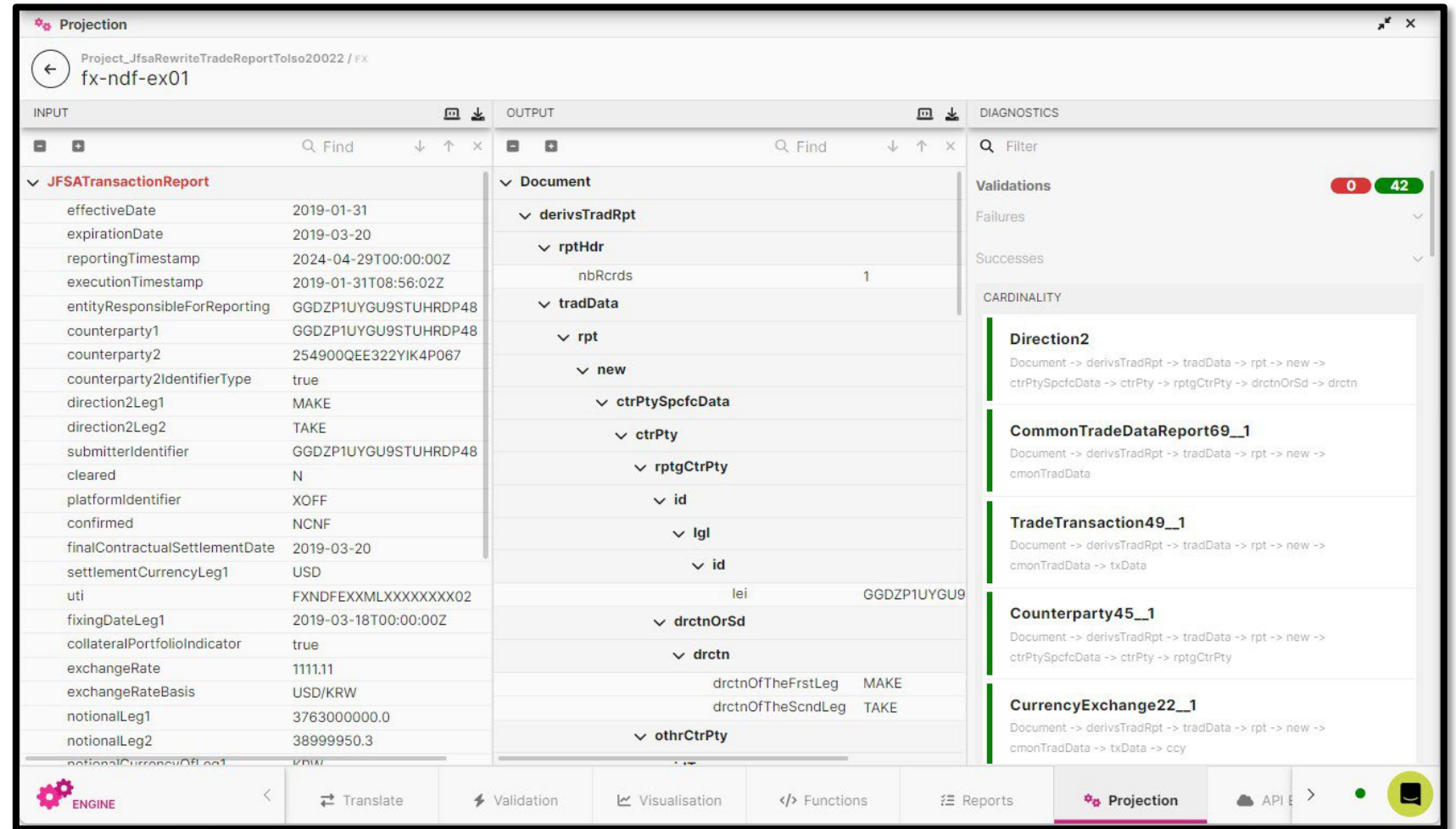
Show Changes Upload Download Fields

Sample	nbRcrds	lei	cd (1)	clrThrsld			
Equity Option Price Return Basket ex01 New	1	DUMMY000000000...	AIFD	true	51	-	-
Equity Option Price Return Basket ex02 Amend	1	DUMMY000000000...	INVF	true	51	-	-
Equity Option Price Return Index ex01 European Call	1	DUMMY000000000...	UCIT	true	59	-	-
Equity Option Price Return Index ex02 European Call	1	213800WWTABZ1G...	UCIT	true	55	-	-
Equity Option Price Return Index ex03 European Call	1	213800WWTABZ1G...	AIFD	true	55	-	-
Equity Option Price Return Index ex04 European Call	1	213800WWTABZ1G...	ORPI	true	55	-	-
Equity Option Price Return Stock ex01 American Call	1	DUMMY000000000...	CSDS	true	54	-	-
Equity Option Price Return Stock ex02 American Call	1	549300O5MFEP1XJ...	INVF	true	54	-	-
Equity Option Price Return Stock ex03 European Call Averaging	1	213800WWTABZ1G...	CDTI	true	56	-	-
Equity Portfolio Swap Price Return Basket ex01	1	TESTHOUSELEI000...	INUN	true	52	-	-
Equity Portfolio Swap Price Return Single Index ex01	1	DUMMYLEI1234567...	CSDS	true	53	-	-

Samples 22 1159

ENGINE Translate Validation Visualisation Functions Reports Projection API

- Similar to Ingestion, the output of the Projection activity can be viewed.
- Both the inbound and outbound data are available, along with diagnostics against the reporting rules defined in the DSL.



The screenshot displays the 'Projection' window for the project 'Project_JfsaRewriteTradeReportTolso20022 / FX' with the identifier 'fx-ndf-ex01'. The interface is divided into three main sections: INPUT, OUTPUT, and DIAGNOSTICS.

INPUT: A table showing the 'JFSATransactionReport' with various fields and their values.

Field	Value
effectiveDate	2019-01-31
expirationDate	2019-03-20
reportingTimestamp	2024-04-29T00:00:00Z
executionTimestamp	2019-01-31T08:56:02Z
entityResponsibleForReporting	GGDZP1UYGU9STUHRDP48
counterparty1	GGDZP1UYGU9STUHRDP48
counterparty2	254900QEE322YIK4P067
counterparty2IdentifierType	true
direction2Leg1	MAKE
direction2Leg2	TAKE
submitterIdentifier	GGDZP1UYGU9STUHRDP48
cleared	N
platformIdentifier	XOFF
confirmed	NCNF
finalContractualSettlementDate	2019-03-20
settlementCurrencyLeg1	USD
uti	FXNDFEXXMLXXXXXXXXXX02
fixingDateLeg1	2019-03-18T00:00:00Z
collateralPortfolioIndicator	true
exchangeRate	1111.11
exchangeRateBasis	USD/KRW
notionalLeg1	3763000000.0
notionalLeg2	389999950.3
notionalCurrencyOfLeg1	KRW

OUTPUT: A hierarchical tree structure showing the output of the projection. The root is 'Document', which contains 'derivsTradRpt', 'rptHdr', 'tradData', 'rpt', 'new', 'ctrPtySpecfcData', 'ctrPty', 'rptgCtrPty', 'id', 'lgl', 'id', 'lei', 'drctnOrSd', 'drctn', 'drctnOfTheFrstLeg', 'drctnOfTheScndLeg', and 'othrCtrPty'.

DIAGNOSTICS: A section on the right showing 'Validations' with 0 failures and 42 successes. Below this, a 'CARDINALITY' section lists several rules with their cardinality counts.

- Direction2**: Document -> derivsTradRpt -> tradData -> rpt -> new -> ctrPtySpecfcData -> ctrPty -> rptgCtrPty -> drctnOrSd -> drctn
- CommonTradeDataReport69_1**: Document -> derivsTradRpt -> tradData -> rpt -> new -> cmonTradData
- TradeTransaction49_1**: Document -> derivsTradRpt -> tradData -> rpt -> new -> cmonTradData -> txData
- Counterparty45_1**: Document -> derivsTradRpt -> tradData -> rpt -> new -> ctrPtySpecfcData -> ctrPty -> rptgCtrPty
- CurrencyExchange22_1**: Document -> derivsTradRpt -> tradData -> rpt -> new -> cmonTradData -> txData -> ccy

The bottom of the window features a navigation bar with buttons for 'ENGINE', 'Translate', 'Validation', 'Visualisation', 'Functions', 'Reports', 'Projection', and 'API'.

- The screenshot displays the 'API Export' application window. On the left, a sidebar lists various services, with 'CME_ClearedConfirm_17-ingestion-service' selected and highlighted. The main area is titled 'DETAILS' and contains information about the 'INGESTION SERVICE'. It explains that the service exports a REST API for ingesting documents like FpML and ISDA Create into the CDM model. A note states that if the model definition changes, the API must be re-exported. Below this, a table provides the 'BASE URL' and 'API KEY'. An 'EXAMPLE' section follows, showing a curl command to invoke the API and a list of parameter definitions.

API EXPORT	
json-validator-service	●
regulation-report-service	●
run-function-service	●
FpML_5_Confirmation_To_TradeState-ingestion-service	●
FpML_5_Confirmation_To_WorkflowStep-ingestion-service	●
ORE_1_0_39-ingestion-service	●
CME_ClearedConfirm_17-ingestion-service	●
DTCC_9_0-ingestion-service	●
DTCC_11_0-ingestion-service	●
CME_SubmissionIRS_1_0-ingestion-service	●
FIS_TRADE-ingestion-service	●

DETAILS

INGESTION SERVICE

Exports a REST API that ingests external documents, such as FpML and ISDA Create, and translates them to the CDM model.

If the model definition in the current workspace is changed, the API must be re-exported to pick up the changes.

The exported API can be invoked by sending a POST request from a REST client, such as Postman or curl, to the **Endpoint URL** with the generated **API Key**.

BASE URL	https://ui.rosetta-technology.io/api/promoted/ingest/system/read-only-COMMON-DOMAIN-MODEL/latest/CME_ClearedConfirm_1_17
API KEY	eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWiiOiJsaWZlZWxfYXV0aDBfNjUwODBINmUxMGI5OGNjNzNhYzQ0Y2Exliwicm9sZXMiOiJibHZHJyIiwiaWF0IjoiMTUxOTUyMTEyMDAifQ.avyvnoETV-3pTMNuNzXCUC2I

EXAMPLE

See below for an example using curl:

```
curl -H "Authorization: <api-key>" \
-H "Content-Type: <content-type>" \
-d "<xml-or-json>" \
<base-url>
```

Where:

 - <api-key> - generated API key.
 - <content-type> - POST content-type (e.g. "application/xml" or "application/json").
 - <xml-or-json> - Input document (e.g. FpML xml or ISDA Create json) to be ingested.
 - <base-url> - REST endpoint URL.

At the bottom of the window, there is a navigation bar with icons for ENGINE, Translate, Validation, Visualisation, Functions, Reports, Projection, and API Export (which is currently active).

F	1	In-bound Data – mapping & synonyms	Complete
	2	Out-bound Data – reports and projection	Complete
	3	Getting Involved	Start
	4	Contributing to the CDM	Start

- We have already seen that the Common Domain Model (CDM) is an open source project, overseen by FINOS.
- FINOS describes itself as “a Community Creating Open Source Solutions for Financial Services, providing an independent setting to deliver software and standards that address common industry challenges and drive innovation.”
- A COMMUNITY OF PEOPLE: Developers, technologists, and industry leaders from the world’s largest financial services and tech firms, collaborating on a wide range of open-source projects.
- A COMMUNITY OF COMPANIES: Financial services firms taking full advantage of open source, becoming strategic contributors, and realizing the enormous benefits of collaboration.
- A COLLABORATION OF CREATIVE MINDS: An unparalleled opportunity to work together in a neutral environment with the companies and talents driving open source in financial services.
- Now that you have a good knowledge of the CDM, let’s look at how **you** can join this community and collaborate on the future development of the CDM.

Here are the key ways to interact with the CDM community:

- Working Groups
 - These are the primary interactive forums for participants to collaborate on the development of the CDM. With defined objectives and topic areas, the Working Groups meet on a regular, defined schedule (details provided on following pages).
- Email
 - FINOS maintains a number of mailing lists to distribute information on the CDM and also to enable collaboration within the community, including the Working Groups.
- GitHub
 - As we have seen, [GitHub](#) is the repository that stores the CDM model.
 - But it is also used for communication, including tracking issues and model updates, and the meeting schedules, agendas and minutes for the Working Groups.
- We will take a look at each of these in more detail.

The **CDM Collateral Working Group** plays a pivotal role in the CDM ecosystem by providing a robust framework for harmonizing data and processes related to collateral flows and documentation.

- Meetings usually take place on the last Tuesday of every month at 10 am EST / 3 pm GMT.
- Email help@finos.org to be added to the meeting invites directly, or find the meeting in the [FINOS Community Calendar](#).
- To subscribe to the collateral working group mailing list and stay updated on meetings and agenda, please visit: <https://lists.finos.org/g/cdm-collateral-wg>.
- You can view copies of the agenda, materials, minutes and action items of the CDM Collateral Working Group meetings on [GitHub](#):
 - [Upcoming meetings](#).
 - [Past meetings](#).

These are the other Working Groups; click on their links for more information, meeting schedule, distribution emails and contact details.

- [CDM Steering Working Group](#)

Serves as the driving force behind the strategic direction and technical aspects of the CDM. Responsible for establishing technical and modeling guidelines, setting and revising the project's strategic roadmap, and vetting proposed changes.

- [CDM Contribution Review Working Group](#)

Reviews, approves, and implements contribution proposals to the CDM, ensuring the integrity and quality of the project. Serve as arbitrators for any disputes arising from contributions, upholding the design principles and guidelines that govern the CDM.

- [CDM Technology Architecture Working Group](#)

Focuses on aligning the architecture, tooling services, infrastructure, development practices, and deployment of the CDM with both existing and future technologies.

- [CDM Derivatives Working Group](#)

Focused on swaps (IR, credit, equity, commodity, etc.), options, FX, and post-trade events (increases, novations, exercises, etc.)

- [CDM Collateral Working Group](#)

Provides a framework for harmonizing data and processes relating to collateral flows and documentation.

F	1	In-bound Data – mapping & synonyms	Complete
	2	Out-bound Data – reports and projection	Complete
	3	Getting Involved	Complete
	4	Contributing to the CDM	Start

- In the previous section, we discussed getting involved in the CDM, through participation in the FINOS-led Working Groups.
- You can also get involved in a much more practical sense – by contributing to the CDM itself and being part of the global, distributed group of collaborators that analyses, designs, builds, tests and releases changes to the model, its definitions and its implementation.
- This is an opportunity that is not open solely to technologists or developers; the core model of the CDM can be understood by practitioners across the financial services industry and everyone's expertise and insights are welcomed. Many of the contributions and improvements are not technical; they represent valuable adjustments to the model's product, event and lifecycle modelling, based on the combined expertise of the participants.
- Having said that, as a shared resource, there are some rules of engagement and some key processes to follow. We will look at those next.

- If you have an idea, suggestion for improvement, fix to an issue, or even just a concern, you can raise it to the community to foster engagement and collaboration.
- To raise an Issue:
 - Navigate to the CDM project on GitHub: <https://github.com/finos/common-domain-model/issues>.
 - Click on “New Issue”.
 - Give a full and complete description and submit your item.
- All participants are encouraged to raise issues and to review and comment on existing open issues raised by others: perhaps you are facing the same or similar issue? Or perhaps you have a suggestion about how the item could be addressed?
- Issues are typically discussed in the CDM Working Groups, so to get focus on your issue, it is recommended to raise it in the Working Group for your domain.
- Have a look at some of the [open issues](#) now. Do you see any where you can make a contribution? Perhaps take a look at the [collateral-related issues](#).

- If you have decided that you want to develop a new feature or make a change or improvement to an existing feature, you may develop and propose it as Contributor using a GitHub Pull Request.
- Before we look at the mechanics of this activity, let's look at this pre-modelling checklist:
 - Review the design-principles and governance (covered in the next pages).
 - Make sure you have the introductory materials for your chosen modelling platform ready; for example, we suggest the Rosetta Starter Guide or equivalent.
- In addition, for large model changes or changes to core data types, it is recommended that you:
 - Define use case: Identify and document one or more use cases with details (e.g. a sample trade).
 - Draft conceptual design (high level): Draft a conceptual view showing the set of data types, their definitions (and/or sample attributes), their relationships to each other, and, if applicable, a workflow.
 - Design approval: Obtain approval of high-level conceptual design from a CDM Working Group or the Contribution Review Working Group.

- The purpose of this section is to detail the CDM design principles that any contribution to the CDM development must adhere to. The CDM supports the market objectives of standardisation via a set of design principles that include the following concepts:
 - Normalisation through abstraction of common components, e.g. price or quantity.
 - Composability where objects are composed and qualified from the bottom up.
 - Mapping to existing industry messaging formats, e.g. FpML.
 - Embedded logic to represent industry processes, e.g. data validation or state-transition logic.
 - Modularisation into logical layers, using namespace organisation.

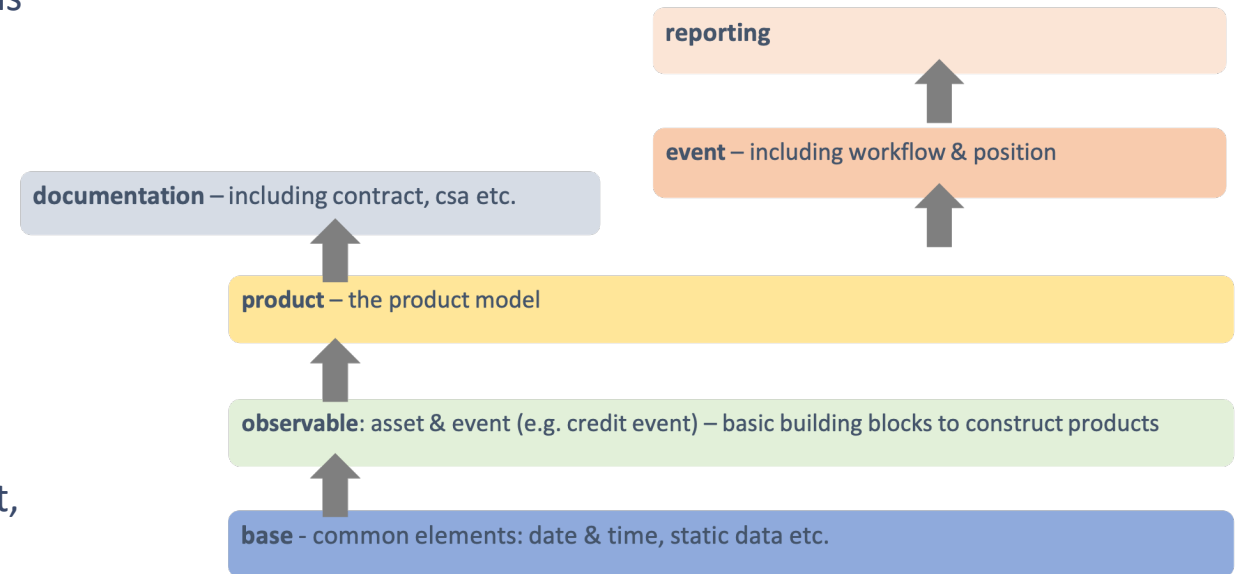
- To achieve standardisation across products and asset classes, the CDM identifies logical components that fulfil the same function and normalises them, even when those components may be named and treated differently in the context of their respective markets. Such design reaffirms the goal of creating an inter-operable ecosystem for the processing of transactions across asset classes.
- An example of this approach is the normalisation of the concepts of quantity, price and party in the representation of financial transactions. The CDM identifies that, regardless of the asset class or product type, a financial transaction always involves two counterparties trading (i.e. buying or selling) a certain financial product in a specific quantity and at a specific price. Both quantity and price are themselves a type of measure, i.e., an amount expressed in a specific unit which could be a currency, a number of shares or barrels, etc. An exchange rate between currencies or an interest rate also fits that description and are represented as prices.
- This approach means that a single logical concept such as quantity represents concepts that may be named and captured differently across markets: e.g. notional or principal amount etc. This, in turn, allows to normalise processes that depend on this concept; for instance, how to perform an allocation (essentially a split of the quantity of a transaction into several sub-transactions) or an unwind, instead of specialised IT systems handling it differently for each asset class.
- It is imperative that any request to add new model components or extend existing ones is analysed against existing components to find patterns that should be factored into common components and avoid specialising the model according to each use case. For instance, in the model for averaging options (often used for commodity products, whereby multiple price observations are averaged through time to be compared to the option's strike price), the components are built and named such that they can be re-used across asset classes.

- To ensure re-usability across different markets, the CDM is designed as a composable model whereby financial objects can be constructed bottom-up based on building-block components. (Remember the building blocks earlier in module C?)
- The main groups of composable components are:
 - Financial products: e.g. the same option component is re-used to describe option payouts across any asset class, rather than having specialised Swaption, Equity Option or FX option, etc., components.
 - Business events that occur throughout the transaction lifecycle are described by composing more fundamental building blocks called primitive events: e.g. a partial novation is described by combining a quantity change primitive event (describing the partial unwind of the transaction being novated away) and a contract formation primitive event (describing the new contract with the novation party).
 - Legal agreements that document the legal obligations that parties enter into when transacting in financial products are constructed using election components associated to functional logic that is re-usable across different types of agreement: e.g. the same logic defining the calculation of margin requirements can be re-used across both initial and variation margin agreements.
- In this paradigm, the type of object defined by the CDM, whether a financial product, business event or legal agreement, is not declared upfront; instead, the type is inferred through some business logic applied onto its constituents, which may be context-specific based on a given taxonomy (e.g. a product classification).

- To facilitate adoption by market participants, the CDM is made compatible with existing industry messaging formats. This means that the CDM does not need to be implemented "wholesale" as a replacement to existing messaging systems or databases but can coexist alongside existing systems, with a translation layer. In fact, the CDM is designed to provide only a logical model but does not prescribe any physical data format, neither for storage nor transport. This means that translation to those physical data formats is built-in, and the CDM is best thought of as a logical layer supporting inter-operability between them.
- The need for such inter-operability is illustrated by a typical trade flow, as it exists in derivatives: a trade may be executed using the pre-trade FIX protocol (with an FpML payload representing the product), confirmed electronically using FpML as the contract representation, and reported to a Trade Repository under the ISO 20022 format. What the CDM provides is a consistent logical layer that allows to articulate the different components of that front-to-back flow.
- In practice, mapping to existing formats is supported by synonym mappings, which are a compact description in the CDM of how data attributes in one format map to model components. In turn, those synonym mappings can support an ingestion process that consumes physical data messages and converts them into CDM objects.
- The CDM recognises certain formats as de-facto standards that are widely used to exchange information between market participants. Their synonym mappings are included and rigorously tested in each CDM release, allowing firms that already use such standards to bootstrap their CDM implementation. Besides, because most standard messaging formats are typically extended and customised by each market participants (e.g. FpML or FIX), the CDM allows the synonym representation for those standards to be similarly inherited and extended to cover each firm's specific customisation.

- The CDM is designed to lay the foundation for the standardisation, automation and inter-operability of industry processes. Industry processes represent events and actions that occur through the transaction's lifecycle, from negotiating a legal agreement to allocating a block-trade, calculating settlement amounts or exchanging margin requirements.
- While FINOS defines the protocols for industry processes in its documentation library, differences in the implementation minutia may cause operational friction between market participants. Even the protocols that have a native digital representation have written specifications which require further manual coding in order to result in a complete executable solution: e.g. the validation rules in FpML, the Recommended Practices/Guidelines in FIX or CRIF for SIMM and FRTB, which are only available in the form of PDF documents.
- Traditional implementation of a technical standard distributed in prose comes with the risk of misinterpretation and error. The process is duplicated across each firm adopting the standard, ultimately adding up to high implementation costs across the industry.
- By contrast, the CDM provides a fully specified processing model that translates the technical standards supporting industry processes into a machine-readable and machine-executable format. Systematically providing the domain model as executable code vastly reduces implementation effort and virtually eliminates the risk of inconsistency. For instance, the CDM is designed to provide a fully functional event model, where the state-transition logic for all potential transaction lifecycle events is being specified and distributed as executable code. Another CDM feature is that each model component is associated with data validation constraints to ensure that data is being validated at the point of creation, and this validation logic is distributed alongside the model itself.

- The set of files that define the CDM data structures and functions are organised into a hierarchy of namespaces.
- All components belong to one of the layers and composition can be used to create new modular components from any of those lower in the hierarchy, as shown here.
- Namespaces have many benefits:
 - Accelerated understanding of the model by allowing users to easily see a high-level view of the model and easily find, select, and study specific areas of interest.
 - Faster and easier to find data types and functions for referencing / use in new components.
 - Allowing for partial adoption of areas of interest in the model.
 - Smaller upgrades representing new versions limited to the name spaces that are impacted.
- When developing new CDM components, the positioning of those components in the namespace hierarchy is critical as part of the design (or potentially the re-organising of the hierarchy following the new development), to ensure the CDM remains well organised.



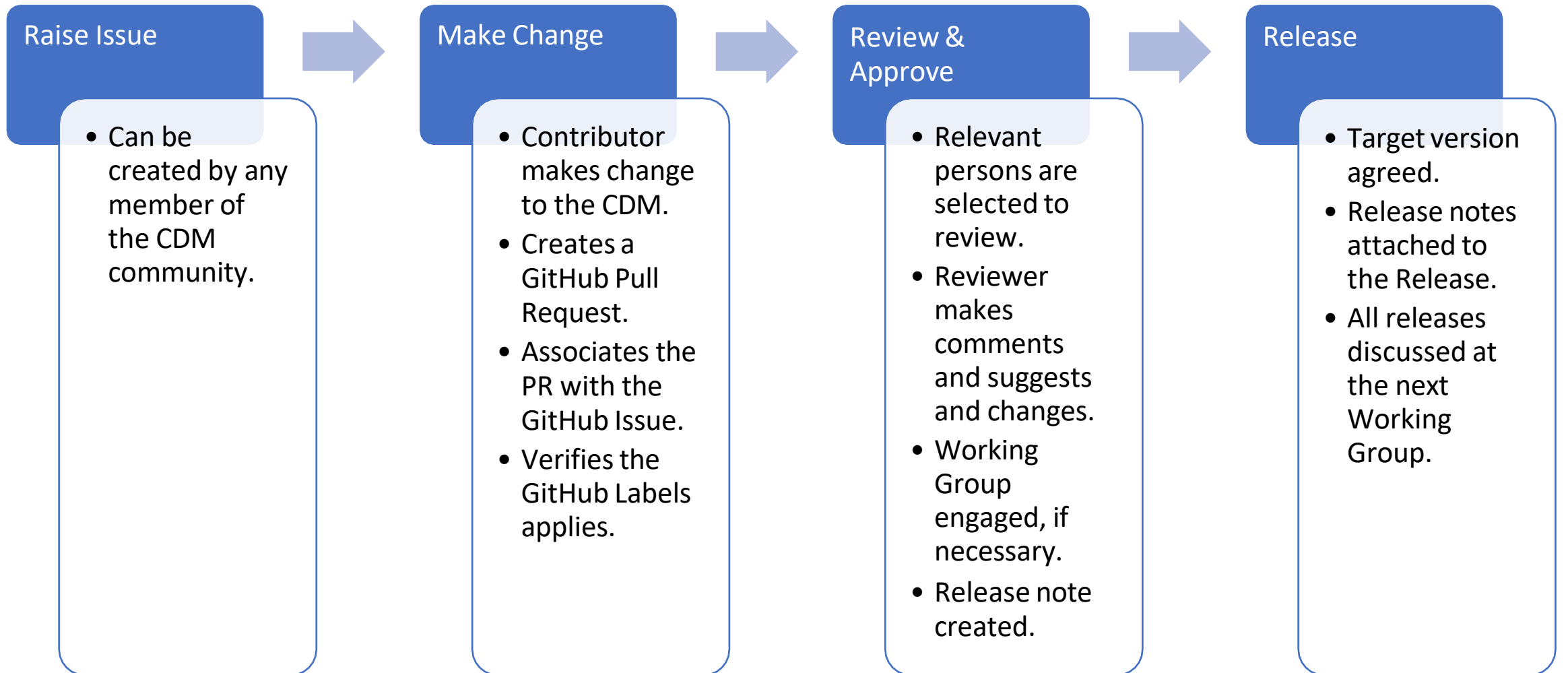
- Several versions of the CDM are available concurrently, with a dual objective.
- The latest **development** version fosters continued, rapid change development and involves model contributions made by the industry community. Changes that break backward compatibility are allowed. This development version is available in read-only and read-write access on the CDM's modelling-platforms.
- The latest **production** version offers a stable, well-supported production environment for consumers of the model. Unless under exceptional circumstances, no new disruptive feature shall be introduced, mostly bug fixes only. Any change shall adhere to a strict governance process as it must be backward-compatible. Generally, it can only be developed by a CDM Maintainer. This production version is available in read-only access through the CDM's modelling-platforms.
- Earlier production versions, when still supported, are also available in read-only access for industry members who are still implementing older versions of the model. Over time, those earlier production versions enter long-term support in which supportability will be degraded, until they eventually become unsupported..

- The three trade associations and FINOS maintain a roadmap of future releases and functionality which can be viewed on GitHub at any time: <https://github.com/finos/common-domain-model/blob/master/ROADMAP.md>
- An example is shown here:

	Title/Topic	Chair	Q1	Q2	Q3	Q4
Working Groups	Steering WG	David Shone, ISDA	- Complete governance updates	- Annual Review of governance- end June - Establish release schedule and process		
	Technical Architecture WG	Chris Rayner, ISLA	- Release build process moves to Github Actions	- Release build process moves to Github Actions -- Redesign Testing Process - Vision Statement - Prioritise initiatives	- Serialisation Phase 1 begins - Ref data list management phase 1/2- TBC	- Serialisation Phase 1 complete - Ref data list management phase 3 TBC
	Contribution Review WG	Rotating- trade associations	- Establish Release management process/bed in resources: Release Manager & Engineer - Release approval & review	- Release approval & review	- Release approval & review	- Release approval & review
	->Cross-product Modelling	N/A	- Product model changes: -- Asset & Observable refactoring [supports structured products and strategic fix for sec finance qualifications] -- Define product with contract details -- Redesign Product Qualification to separate economic qualification from product qualification	- Product model changes: -- Asset & Observable refactoring [supports structured products and strategic fix for sec finance qualifications]	- Product model changes: -- Harmonise date/timestamp	
	Collateral WG	Vernon Alden-Smith, ISDA	- Extend ECS model - Repo collateral extension	Ongoing adoption support & WG prioritised items		
	Securities Lending WG	Chris Rayner, ISLA	Ongoing adoption support and WG prioritised items			
	Derivatives Product and Business Event WG	David Shone, ISDA	- Migrate to FINOS governance umbrella - Option payout refactoring (ETD/OTC) - Product enhancements driven by DRR - Member modelling proposals: Equity Swaps	- Member modelling proposals	- Member modelling proposals	- Member modelling proposals
	Structured Products WG	Jean-Baptiste Ziade, Frangos Chain	Ongoing structured product enhancements & WG prioritised items			
	ICMA Repo/Bonds WG	Gabriel Callsen, ICMA	Ongoing adoption support & WG prioritised items			
	Securities Finance Reg Reporting?	TBC				
	ISDA Legal Agreement WG	Vernon Alden-Smith, ISDA Ciaran McGonagle, ISDA	- Analyse and develop framework for remaining 20 clauses of legacy CSA agreements - Model 10 clauses of legacy CSAs analysed in 2023	Analyse and develop framework for remaining 20 clauses of legacy CSA agreements	Complete legacy CSAs in CDM and create test data	Complete legacy CSAs in CDM and create test data
	ISDA DRR	Eleanor Hsu, ISDA Tabish Ahmed, ISDA	- DRR 4.0 Release: Complete coverage EMIR and JFSA - FCA dev complete target 30th March	- EMIR /JFSA Compliance Dates [1st/29th April] - ASIC & MAS dev complete target	- FCA compliance date- 30th Sept	- ASIC/MAS Compliance date- 21st October

- Before you start contributing your model changes, please go through the following contribution checklist:
 - Specify a meaningful title and description for the contribution.
 - Notify the CDM Maintainers (via email or Slack) of the submitted contribution.
 - Include: Any notes on expected mapping, validation or qualification changes; additional data samples; documentation adjustment; and release notes.
- Changes are contributed by submitting a Pull Request for review into the CDM source-control repository. This Pull Request will invoke a build process to compile and run all CDM unit tests and regression tests.

- All proposed changes go through the same governance process:



F

1

In-bound Data – mapping & synonyms

[Review](#)

2

Out-bound Data – reports and projection

[Review](#)

3

Getting Involved

[Review](#)

4

Contributing to the CDM

[Review](#)

Well Done! You have completed the final module.
You can review any of the complete sections again.

[Done](#)